



Be a Rebel...

But Trust Me on This: Why **ColdBox** Is Worth a Look

The Hesitant Developer's Guide to ColdBox

Into The Box 2026

"For developers who've avoided frameworks since the Bush administration."





Senior ColdFusion Developer

Michael Rigbsy



@mike-rigsby



mrigsby@gmail.com



@mrigsby



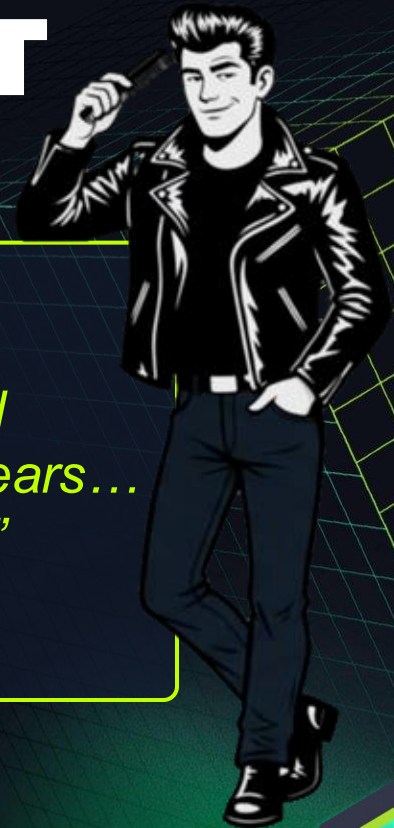
@michael.rigsby.9

- Developing ColdFusion applications since ~1999
- Reformed ColdBox hesitant developer ;-)
- Frequent contributor to CBWire
- I have an office staff that consists of two Dogs and a Cat





THE REBEL MINDSET



"I don't need a framework"

- Independence as a badge of honor
- Legacy code familiarity vs. fear of the unknown
- The myth: "Framework = loss of control"
- Years of muscle memory and tribal knowledge
- "If it works, don't touch it" culture

"If you've avoided ColdBox for 20 years... that's dedication."



ColdBox has conventions for everything... handlers, models, views. Now you can stop pretending your utils folder is 'organized'



THE PAIN WE ALL KNOW



Messy routing & ad-hoc URL handling



Scattered business logic across .cfm files



“Where does this even go?” architecture



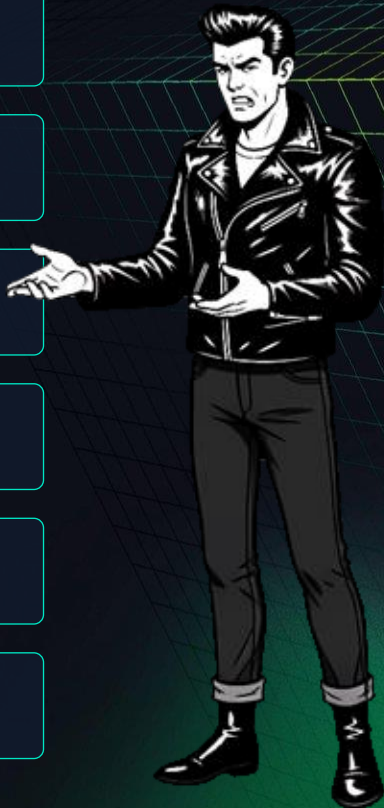
Brittle legacy patterns that break when touched



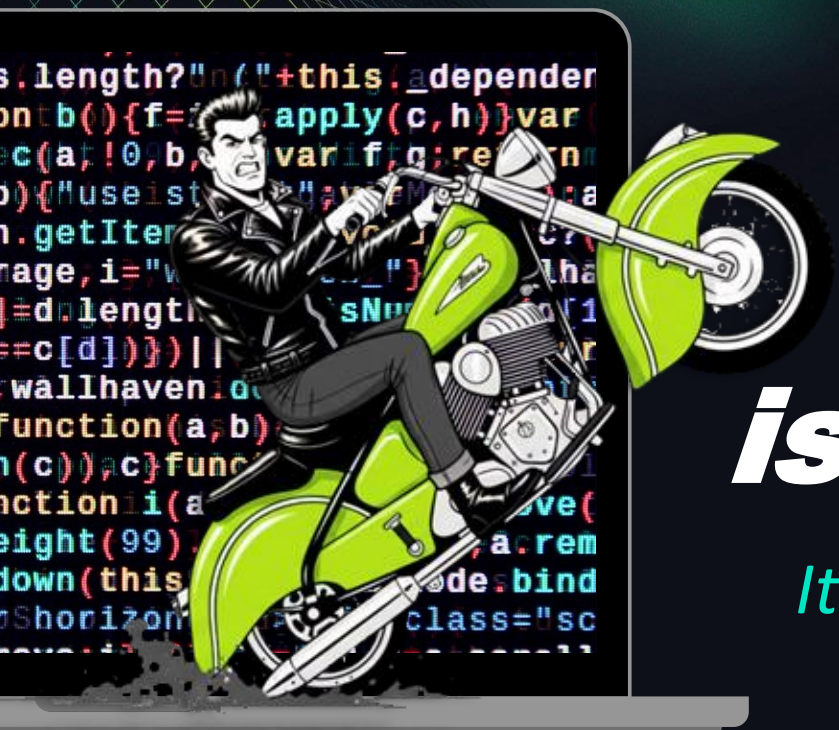
Custom utilities you shouldn't have to still maintain in 2026



Debugging spaghetti code at 2 AM



If your app has a file named “utils.cfm”, this session is probably for you.



CHAOS

is a framework

It's just an undocumented one



Every app has structure. The question is whether it's intentional.



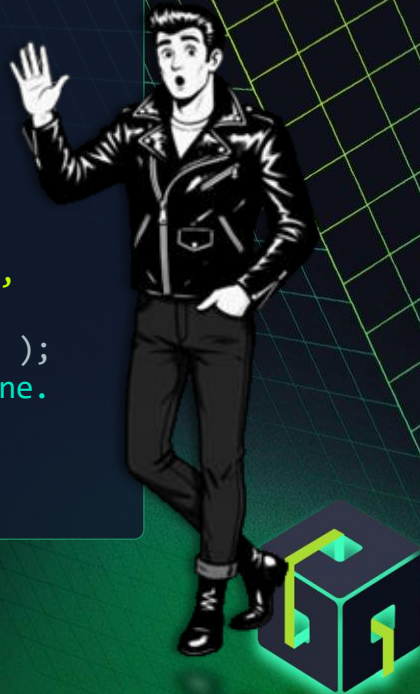
ROUTING: BEFORE & AFTER

WITHOUT COLDBOX

```
<!--- ComplicatedPageRouting.cfm --->
<cfif url.action EQ "view">
    <cfinclude template="pages/view.cfm">
<cfelseif url.action EQ "edit">
    <cfinclude template="pages/edit.cfm">
<cfelseif url.action EQ "delete">
    <cfinclude template="pages/delete.cfm">
<cfelseif url.action EQ "save">
    <cfinclude template="pages/save.cfm">
</cfif>
<!--- 47 more conditions... --->
```

WITH COLDBOX

```
// config/Router.cfc
route( "users/:id" )
    .withAction({
        GET      : "show",
        POST     : "save",
        PUT      : "update",
        DELETE   : "remove"
    }).toHandler( "users" );
// Clean. Predictable. Done.
```



Before ColdBox, my routing strategy was `index.cfm?do=something&please=work`



ONE FRAMEWORK, YOUR CHOICE OF LANGUAGE

ColdBox 8 : multiple languages on the same framework

- ✓ Classic CFML/CFScript CF developers already know
- ✓ Mixed CF and BoxLang on the BoxLang Multi-runtime Architecture
- ✓ BoxLang Prime (bx-coldbox) The pre-compiled, natively optimized version of ColdBox that runs without the CFML compatibility layer at all

Why this matters for the skeptics in the room

- ✓ No forced migration, your existing .cfm and .cfc files run today, as-is
- ✓ Gradual modernization path. Start in CFML and move to BoxLang at your own pace, or mix both in the same project
- ✓ BoxLang's dual parser runs CFML code natively, so it can serve as a drop-in engine replacement for Adobe CF or Lucee
- ✓ BoxLang Prime unlocks native JVM compilation and performance gains for teams ready to go all-in



ColdBox doesn't force you to pick a lane... it gives you every lane and lets you merge when you're ready.



COLDBOX IN 60 SECONDS



MVC Convention Framework

Handlers, Views, Models — everything has a home



Module Ecosystem

WireBox, LogBox, CBSecurity, CBWire and 200+ FORGEBOX packages



CommandBox Powered

CLI scaffolding, package management, server management, task runners



Test-First Ready

TestBox integration for unit, integration, and BDD testing



20 Years Battle-Tested

Production-proven across thousands of CFML applications



ColdBox comes with CacheBox, LogBox, and WireBox. My old app came with PanicBox



THE **COLDBOX** ECOSYSTEM

Entire categories of code you never have to write again.



WireBox

Dependency injection & auto-wiring

No more manual object creation



LogBox

Enterprise-grade logging

Configurable appenders & levels



CBSecurity

Auth, authorization & rules

Stop rolling your own auth



TestBox

BDD & TDD testing framework

Unit, integration & spec testing



cbValidation

Object & form validation

Declarative constraint rules



CBWire

Reactive applications in record time

Revolutionizes your web development



Before WireBox, my idea of dependency injection was copying and pasting the same createObject() call into 47 files



SECURITY: STOP ROLLING YOUR OWN



Authentication

Pluggable auth providers, session management, JWT support

Authorization

Role-based and permission-based access control

Security Rules

Declarative rules via config, not scattered cfif blocks

CSRF Protection

Built-in token generation and validation

Security Headers

Automatic OWASP-recommended response headers

Single Sign-On (SSO)

Integrate SSO into your application easily with CBSSO

Passkeys

Integrate SSO into your application easily with cbsecurity-passkeys



TESTS: THE CASE YOU'RE IGNORING

TESTING WITHOUT STRUCTURE

```
<!-- Question: How do you test this? -->
<cfinclude template="process.cfm">
<!-- Mixed DB + logic + output -->
<!-- Answer: you don't. -->
```



TESTING WITH COLDBox + TESTBOX

```
it( "should return a user", function(){
  var event = execute("users.show");
  expect( event.getValue("user") ).toBeStruct();
});
```



Isolated business logic is testable business logic



Integration tests verify your entire request lifecycle



BDD specs serve as living documentation



CI/CD pipelines become possible, not aspirational



ColdBox gives you handlers, models, and views. My old app had handlers, prayers, and blame.



COLDBOX PLAYS NICE WITH LEGACY

You don't have to burn your app down to build something better.

1

Coexistence

ColdBox runs alongside your existing .cfm files

2

Incremental Routes

Move one URL pattern at a time into ColdBox routing

3

Service Extraction

Pull business logic into models without rewriting templates

4

Module Isolation

New features get built as ColdBox modules from day one

5

Strangler Pattern

Legacy code shrinks naturally as new code grows



The ColdBox config file is ColdBox.cfc. My old config file was called 'that one query at the top of Application.cfm that nobody should touch'



CONVENTIONS REDUCE DECISION FATIGUE



handlers/

Request handling & controller logic



modules/

Self-contained features & packages



models/

Business logic & services



config/

Routes, settings, environments



views/

Presentation & UI templates



tests/

Unit, integration & BDD specs



They say convention over configuration saves time. You know what doesn't save time? Debugging your coworker's custom framework from 2007



COMMANDBOX: THE GATEWAY

Even if you never adopt ColdBox, CommandBox will change your workflow.

Package Management

`install coldbox / install cbsecurity`

Server Management

`server start / server stop` – no more servlet configs

Scaffolding

`coldbox create app / coldbox create handler`

Environment Configs

Per-environment settings, `.env` support

REPL

Test expressions and snippets instantly



I installed ColdBox with CommandBox in 30 seconds. My last framework migration took 30 months. And a therapist



COMMANDBOX: THE GATEWAY

Task Runners

“Task runners are the next generation of automation for CFML developers. Now you don't need to learn another tool to improve your workflow. You can automate directly in CFML!”

Write Your Own Module (and share it on ForgeBox!)

Need to extend CommandBox beyond its built-in functionality? You can write your own module. If you've worked with ColdBox modules, you'll find the architecture familiar. CommandBox modules follow a very similar structure and convention set

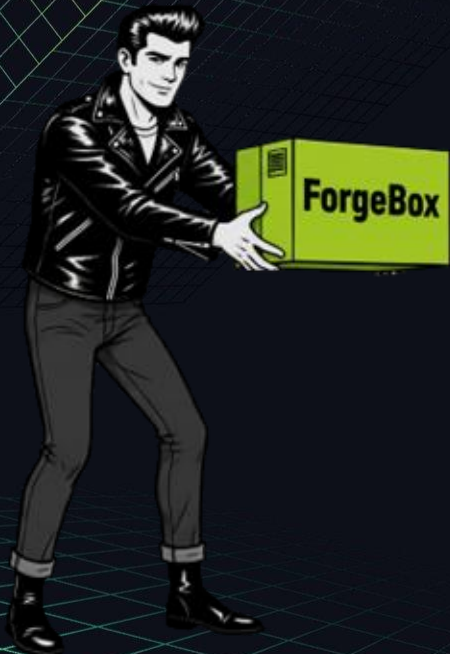
<ShamelessPlug>

Two of my CommandBox modules on ForgeBox.io

DNS Utilities : forgebox.io/view/dns-utility

CBWire CLI : forgebox.io/view/cbwire-cli

</ShamelessPlug>



Before ColdBox, my application architecture was best described as 'archaeological.' You just kept digging and finding older code



MIGRATION WITHOUT MUTINY

TECHNICAL STRATEGY

- Start with routing , the biggest bang for buck
- Add WireBox for one service
- Move one business process to a handler
- Build the next feature as a module
- Add tests as you go, not all at once

TEAM STRATEGY

- Don't announce a "migration", just improve
- Let early wins build momentum
- Pair experienced + new devs on ColdBox tasks
- Document conventions as you adopt them
- Celebrate reduced debugging time publicly



The word "rewrite" starts wars. The word "improve" starts progress.



EVEN REBELS HAVE A TEAM



Predictable Structure

New devs find things in minutes, not days



Reduced Tribal Knowledge

The framework is the documentation



Easier Code Reviews

Everyone speaks the same structural language



Shared Conventions

No more “ask Dave, he wrote that part”



Faster Onboarding

Convention-driven code is self-explanatory



Compliance & Audits

Structured code satisfies auditors



ColdBox won't judge your legacy code. I might, but ColdBox won't.



WHAT COLDBOX IS **NOT**



Not a rewrite mandate

Adopt incrementally, one route, one module, one feature at a time



Not a rigid cage

Conventions guide you, they don't trap you



Not all-or-nothing

Use the pieces that solve your problems, ignore the rest



Not a magic wand

It's a power tool, you still have to use it well



My legacy app has a separation of concerns: I write the code, and then I'm concerned



THE **REBEL'S** PAYOFF

- ✓ More control over your codebase, not less
- ✓ Cleaner code that doesn't require archaeology to understand
- ✓ Faster onboarding. New devs ship in days, not months
- ✓ Fewer 2 AM surprises
- ✓ A codebase that doesn't make the next developer cry

ColdBox doesn't ask you to stop being a rebel.



ColdBox: because your Application.cfc shouldn't need its own table of contents

YOUR NEXT 3 MINUTES



1

box install coldbox

Install ColdBox via CommandBox

2

coldbox create app myApp

Scaffold a hello-world application

3

server start

Run it and see what you've been missing



If you hate it, you've lost 3 minutes. If you don't... welcome to the rebellion.



Q&A

War stories welcome.

Migration fears addressed.

Experimentation encouraged.

RESOURCES

```
var resources = {  
  "coldbox.org"      : "All Things ColdBox",  
  "ortusbooks.com"   : "All the box docs",  
  "cfcasts.com"      : "Bite-size video tutorials from the box gurus",  
  "community.ortusolutions.com" : "community forum for CFML & BoxLang developers",  
  "ortusolutions.com" : "Training & Support",  
  "forgebox.io"      : "Module Marketplace",  
  "apidocs.ortusolutions.com" : "Dig even deeper!"  
}
```



Modules let you encapsulate features into reusable packages. I also had reusable packages. I called them 'that zip file Steve emailed me in 2014'